

CYA N

Choose Your AI - Noob

Documentazione Tecnica Architettuale

Versione 7.1.0 (Sviluppo)

Progetto Multi-Agent

Agosto 2026

Indice

1	Panoramica Architetturale e Vincoli di Sistema	3
1.1	Definizione del Sistema	3
1.2	Requisiti di Resilienza e Operatività Offline	3
1.3	Vincoli Hardware e Scelte Progettuali	3
2	Topologia del Sistema e Grafi di Instradamento	5
2.1	Mappatura delle Relazioni Gerarchiche	5
2.2	Struttura del Filesystem e Archi di Dipendenza	5
3	Motore di Instradamento LLM e Smistamento Multi-Dominio V7.1	7
3.1	Ottimizzazione Preventiva: Il Filtro di Complessità	7
3.2	Fase 0: Classificazione Neurale (LLM Router)	7
3.3	La Guardia P0: Mitigazione del Semantic Bleed	8
3.4	Fallback Testuale: Meccanismo di Tolleranza ai Guasti	8
4	Esecuzione della Pipeline Sequenziale in Ambienti a Memoria Vincolata	9
4.1	Architettura Draft & Merge: Esecuzione Asincrona	9
4.2	Sincronizzazione Attiva e Gestione dello Scaricamento (RAM)	9
4.3	Passaggio di Consegne (Handoff) e Salvaguardia del Contesto	10
5	Chat History e Persistenza Conversazionale (V6.6.0)	11
5.1	Architettura della Memoria di Sessione	11
5.2	Meccanismo di Sliding Window	11
5.3	Integrazione e Iniezione del Contesto	11
5.4	Integrità dello Stato e Controllo Utente	12
6	Domain Retention e Sticky Routing (V7.2)	13
6.1	Motivazione Architetturale	13
6.2	Stato di Sessione: last_active_domain	13
6.3	Logica Decisionale: should_sticky_route()	13
6.4	Parametri di Configurazione e Fallback	14
7	Critic Pass, Analisi Strutturata e Sanificazione dell'Output	15
7.1	Revisione Critica: Validazione Antagonistica (Critic Pass)	15
7.2	Parsing Strutturato e Gestione dei Tag di Ragionamento	15
7.3	Intercettazione delle Eccezioni a Basso Livello	16
8	Topologia della Flotta LLM e Parametri di Inferenza	17
8.1	Provisioning e Gerarchia dei Modelli	17

8.2 Calibrazione delle Temperature (Determinismo vs Entropia)	17
9 Istruzioni Operative: Messa in Produzione	19
9.1 Inizializzazione del Motore di Inferenza (Ollama)	19
9.2 Provisioning della Flotta: Scaricamento dei Modelli	19
9.3 Configurazione dell'Ambiente e Avvio	19
10 Glossario Architetturale	20

Capitolo 1

Panoramica Architetture e Vincoli di Sistema

1.1 Definizione del Sistema

CYA N (Choose Your AI - Noob) è un orchestratore intelligente per Large Language Models (LLM), progettato specificatamente per operare in modo esclusivo all'interno di ambienti di calcolo locali. Alla ricezione di un input testuale, il sistema non effettua un semplice inoltro (passthrough) della richiesta verso un modello linguistico generico, ma ne analizza la semantica per determinare un percorso di instradamento ottimale.

La classificazione della query è affidata al modulo `llm_router.py`, che sfrutta un micro-LLM (`qwen2.5:1.5b` via Ollama) come router semantico. Il router restituisce un'etichetta di dominio strutturata in formato JSON, comprensiva dei punteggi di confidenza per i quattro domini base (coding, math, rights, general), un indice di difficoltà (1-3) e un flag `is_followup` per la gestione del contesto multi-turno.

Qualora il router LLM non sia disponibile, il sistema attiva un arco di fallback preventivo basato su un motore di ricerca a parole chiave integrato da algoritmi di approssimazione testuale (Soft-Match) per garantire tolleranza agli errori di battitura. Il nucleo avanzato del progetto risiede nella sua capacità di elaborare interrogazioni interdisciplinari: l'architettura supporta flussi di esecuzione multi-dominio tramite una "Pipeline Sequenziale", permettendo a più agenti specializzati di collaborare alla risoluzione di una singola richiesta complessa.

1.2 Requisiti di Resilienza e Operatività Offline

Un requisito non negoziabile di CYA N è la sua totale indipendenza dalle infrastrutture cloud o di rete esterne. Il software instaura processi operativi che garantiscono il 100% delle funzionalità in modalità offline. Ogni scambio di dati — dalla comunicazione interna tra i moduli di instradamento e il motore di inferenza, fino alla generazione dell'output finale — viene processato interamente all'interno dell'ambiente hardware locale. Questo approccio architetturale riduce a zero la latenza di rete e offre una garanzia assoluta in termini di privacy e sicurezza dei dati processati.

1.3 Vincoli Hardware e Scelte Progettuali

Lo sviluppo di CYA N ha richiesto l'adattamento dell'infrastruttura a un vincolo hardware rigoroso: garantire un'esecuzione stabile su macchine di livello consumer dotate di soli 8

GB di memoria RAM. Per soddisfare questo requisito, l'architettura adotta specifiche soluzioni ingegneristiche di ottimizzazione:

- **Agenti Stateful (Pre-istanziamento):** Al fine di ottimizzare i tempi di risposta, le istanze dei modelli non vengono distrutte e ricreate a ogni ciclo di esecuzione. Esse mantengono uno stato temporaneo in memoria (keep-alive), riducendo le operazioni di lettura (I/O) sul disco e predisponendo l'architettura per la gestione dello storico conversazionale (Chat History).
- **Router LLM a Bassa Impronta Computazionale:** L'analisi semantica è demandata al micro-modello `qwen2.5:1.5b` operante come classificatore dedicato (`llm_router.py`). La sua ridotta impronta in memoria (circa 1 GB) permette di calcolare la classificazione senza saturare la RAM, preservando le risorse per la successiva fase di generazione del testo. Il router viene esplicitamente scaricato (unload) prima dell'attivazione dei modelli generativi.
- **Monitoraggio Dinamico e Sincronizzazione Attiva:** Il sistema esegue controlli preliminari prima di allocare memoria. Tramite la libreria `psutil`, viene interrogato lo stato dell'hardware; in caso di memoria insufficiente, il sistema effettua un declassamento (downgrade) preventivo verso un modello generativo più leggero. Nelle esecuzioni multi-agente, un meccanismo di polling attivo sospende momentaneamente il processo, attendendo che la RAM del primo agente venga liberata prima di inizializzare il secondo, azzerando le probabilità di interruzioni per Out of Memory (OOM).

Capitolo 2

Topologia del Sistema e Grafi di Instradamento

2.1 Mappatura delle Relazioni Gerarchiche

Il flusso di controllo logico di CYA N segue la topologia di un grafo orientato. Il router LLM agisce come nodo di smistamento principale, instradando l'elaborazione verso un singolo nodo specializzato (esecuzione mono-dominio) o rilevando un intento composito che necessita l'attivazione dell'esecuzione multi-agente (Pipeline). Tuttavia, all'attivazione della Pipeline, i parametri generati dal router vengono subordinati a una gerarchia statica. L'ordine di esecuzione degli agenti è imposto rigorosamente dalla matrice di configurazione `pipeline_order_matrix`. Questa struttura dati garantisce la coerenza logica delle risposte: stabilisce, ad esempio, che l'interazione tra i domini Diritto e Programmazione debba sempre fluire dalle direttive normative verso l'implementazione del codice (Rights → Coding), imponendo che sia il vincolo legislativo a dettare i parametri per il software. Inoltre, l'architettura implementa una regola di sbarramento (P0 Guard) che annulla le pipeline ibride comprendenti il dominio generalista, per prevenire deviazioni semantiche.

2.2 Struttura del Filesystem e Archi di Dipendenza

Per assicurare scalabilità e manutenibilità, il progetto implementa una rigorosa separazione delle responsabilità (Separation of Concerns). Ogni modulo sorgente espone interfacce ben definite e instaura dipendenze unidirezionali:

- **main.py (Modulo di Esecuzione e Coordinamento):** Rappresenta l'entry point dell'applicazione (CLI). Gestisce il loop di interazione con l'utente e coordina la logica di instradamento richiamando il router LLM o il fallback keyword. È responsabile dell'orchestrazione della logica Draft&Merge per gli agenti multipli, dell'applicazione della P0 Guard, dell'isolamento della chat history sui domain switch e della gestione sicura delle eccezioni hardware (ResourceExhaustedError).
- **config.py (Configurazione Centralizzata):** Agisce come singola fonte di verità statica per il sistema. Definisce le costanti hardware (soglie di RAM), la configurazione dei modelli (LLM primari, di fallback e temperature), i parametri della pipeline, il modello router e i limiti dinamici per il troncamento del contesto testuale.
- **llm_router.py (Router Semantico LLM):** Modulo di classificazione basato su micro-LLM. Implementa la funzione pubblica `predict()`, che invoca `qwen2.5:1.5b` tramite Ollama e restituisce una tupla `(class_id,`

`confidence, domain_scores, difficulty, is_followup`). Definisce le costanti pubbliche `DOMAIN_NAMES` e `PIPELINE_CLASSES` che garantiscono compatibilità con la futura implementazione del classificatore neurale.

- **neural_classifier.py (Architettura Classificatore Neurale — In sviluppo):** Drop-in replacement pianificato per `llm_router.py`. Implementerà un MLP su encoder frozen (`paraphrase-multilingual-MiniLM-L12-v2`) con testa di dominio (multi-label, 4 output) e testa di difficoltà (3 classi). Espone la stessa interfaccia pubblica di `llm_router.py`.
- **dispatcher_request.py (Sistema di Fallback Testuale):** Subentra qualora il router LLM non sia disponibile (`class_id == -1`). Esegue una classificazione in due fasi: una prima elaborazione ad alta velocità tramite analisi di insiemi (Hard-Match) supportata da espressioni regolari, seguita da un'elaborazione tollerante (Soft-Match) che sfrutta la distanza di Levenshtein per correggere dinamicamente gli errori sintattici.
- **ai_engine.py (Motore di Inferenza e Contesto):** Gestisce il ciclo di vita dei modelli linguistici locali. Definisce la classe astratta `BaseAI` e le sue specializzazioni. Implementa la routine principale di generazione testuale, che gestisce lo streaming in tempo reale da Ollama, il parsing dinamico dei tag strutturati (es. `<think>`) e il monitoraggio preventivo della memoria tramite soglie di tolleranza.
- **prompts_templates.py (Gestore dei Modelli Istruttivi):** Centralizza la definizione del comportamento degli agenti (System Prompts). Mantiene inoltre i template dinamici (Few-Shot) richiesti per la costruzione del contesto operativo e per l'esecuzione dei compiti specifici della pipeline, come il passaggio di consegne (Handoff) e la revisione critica (Critic Pass).
- **helper.py (Libreria di Utilità e Sanificazione):** Modulo dedicato al raffinamento dell'output testuale. Utilizza routine basate su regex per sopprimere tag di sistema e caratteri non supportati. Implementa un dizionario di conversione per trasformare la sintassi matematica strutturata (LaTeX) in caratteri Unicode leggibili, gestendo l'interfaccia utente asincrona tramite un indicatore di caricamento (Spinner) in thread separati.
- **db_query.py (Sorgente di Dati Linguistici):** Contiene le strutture dati predefinite utilizzate per il training del classificatore neurale. Definisce le liste di frasi specifiche per ogni dominio (`INTENT_SENTENCES`) e il dizionario `BRIDGE_SENTENCES` per le query ibride.
- **build_dataset.py (Generatore di Dataset):** Script ausiliario per la costruzione e l'esportazione del dataset di training. Aggrega i dati da

`db_query.py` e li prepara nel formato richiesto per l'addestramento del classificatore neurale.

Capitolo 3

Motore di Instradamento LLM e Smistamento Multi-Dominio V7.1

3.1 Ottimizzazione Preventiva: Il Filtro di Complessità

L'inizializzazione e l'esecuzione di due modelli linguistici distinti rappresenta un processo computazionalmente oneroso, giustificabile unicamente in presenza di richieste ad alta complessità analitica. Per salvaguardare l'efficienza dell'infrastruttura, il sistema esegue una convalida preliminare della profondità della query tramite un Filtro di Complessità. Se il numero di parole dell'input risulta inferiore alla soglia parametrizzata (`min_words_for_pipeline`, default 12), il dispatcher previene l'attivazione dell'esecuzione multi-agente, confinando l'elaborazione al dominio primario individuato. Esiste inoltre un meccanismo di promozione score-based: qualora il router LLM classifichi una query come mono-dominio ma i propri score interni indichino un secondo dominio tecnico con punteggio $\geq$ `pipeline_score_min` (default 0.15) e la difficoltà rilevata sia ≥ 2 , la pipeline può essere promossa dinamicamente.

3.2 Fase 0: Classificazione Neurale (LLM Router)

Il sistema di routing della Versione 7.1.0 sostituisce il motore k-NN con un approccio LLM-as-router. Il modulo `llm_router.py` invoca il micro-modello `qwen2.5:1.5b` come classificatore semantico dedicato, operando a temperatura 0 (inferenza deterministica) con contesto di 4096 token.

Il router riceve la query corrente, l'ultimo dominio attivo (`last_active_domain`) e le ultime query della sessione. Restituisce un oggetto JSON strutturato con i seguenti campi:

- **domain:** l'etichetta più probabile tra le 7 disponibili (coding, math, rights, general, math->coding, rights->coding, rights->math).
- **scores:** dizionario con probabilità per i 4 domini base (somma = 1.0).
- **difficulty:** intero 1–3 (semplice / media / complessa).
- **is_followup:** booleano che indica se la query è un follow-up diretto della risposta precedente.

La funzione pubblica `predict()` restituisce la tupla `(class_id, confidence, domain_scores, difficulty, is_followup)`. I `class_id` 0–3 corrispondono ai

domini mono-dominio; i `class_id` 4-6 identificano le pipeline (`PIPELINE_CLASSES`: 4→math→coding, 5→rights→coding, 6→rights→math).

Il router viene scaricato esplicitamente (`unload_router()`) prima del caricamento dei modelli generativi, liberando la RAM necessaria per l'inferenza principale.

***Nota — Interfaccia Drop-in:** Le costanti pubbliche `DOMAIN_NAMES` e `PIPELINE_CLASSES` di `llm_router.py` sono identiche a quelle dell'architettura neural classifier pianificata (`neural_classifier.py`). La sostituzione futura del router avverrà senza modifiche a `main.py`.*

3.3 La Guardia P0: Mitigazione del Semantic Bleed

Una delle vulnerabilità intrinseche ai motori di classificazione semantica è nota come Semantic Bleed (sovrapposizione semantica): lemmi del linguaggio comune possiedono vettori che possono intersecare i domini strettamente tecnici, generando valutazioni ibride falsate. Al fine di prevenire deviazioni sistemiche (allucinazioni di dominio) e un conseguente consumo improprio di memoria RAM, l'architettura applica un vincolo definito [P0 GENERAL GUARD].

Durante l'elaborazione del routing, qualora il dominio generalista (`general`) figuri come parte integrante della coppia ibrida selezionata, il sistema intercetta la discrepanza, annulla la transizione multi-agente ed esegue un declassamento forzato. La regola di risoluzione è rigorosa: se `general` è il dominio primario (`domain_a`), la richiesta viene gestita interamente da `general` in modalità mono-dominio. Se `general` è il dominio secondario (`domain_b`), esso viene scartato e l'esecuzione viene affidata esclusivamente al dominio tecnico primario.

3.4 Fallback Testuale: Meccanismo di Tolleranza ai Guasti

Nell'eventualità in cui il router LLM non sia disponibile (Ollama offline, timeout o errore di parsing — condizione segnalata da `class_id == -1`), la richiesta viene deviata verso il dispatcher testuale (`dispatcher_request.py`), strutturato per operare in due fasi sequenziali:

- **Analisi Rigorosa (Fase 1 - Hard-Match):** Viene eseguita una ricerca per intersezione di insiemi (set) supportata da una valutazione tramite espressioni regolari. La regex impiegata (`[a-zA-Z0-9 +#]+`) garantisce l'estrazione sicura di token tecnici complessi (es. "C++", "C#"), preservando l'integrità dei simboli speciali. Per limitare la percentuale di falsi positivi, l'algoritmo identifica ed esclude le parole chiave sovrapponibili appartenenti a più domini simultaneamente (`SHARED_TECH`).

- **Analisi Tollerante (Fase 2 - Soft-Match):** Per i token non riconosciuti nella prima fase, il sistema calcola la distanza di Levenshtein rispetto al dizionario di addestramento. Viene implementata una tolleranza dinamica proporzionale alla lunghezza della stringa analizzata, consentendo di associare con precisione termini parzialmente scorretti o recanti errori di battitura.

Capitolo 4

Esecuzione della Pipeline Sequenziale in Ambienti a Memoria Vincolata

4.1 Architettura Draft & Merge: Esecuzione Asincrona

Qualora il router LLM confermi l'identificazione di una richiesta ibrida e la regola di sbarramento (P0 Guard) validi l'assenza di interferenze dal dominio generalista, l'orchestratore sospende la logica a singola istanza per inizializzare la Pipeline Sequenziale.

Questo paradigma si fonda su un'esecuzione sequenziale a due fasi. Il primo LLM ad essere invocato (designato come Agente A dalla `pipeline_order_matrix`) riceve un prompt direzionale restrittivo (Directional Prompt). Le istruzioni impongono l'inibizione di qualsiasi formula di cortesia o conclusione deduttiva: l'agente è incaricato esclusivamente di generare un elaborato tecnico intermedio (Draft). Questa bozza informativa è strutturata non per la lettura umana, ma per essere elaborata programmaticamente dal nodo successivo. L'intera fase generativa intermedia viene occultata all'utente finale tramite un'interfaccia di caricamento asincrona (SpinnerContext), mantenendo pulito l'output a terminale.

4.2 Sincronizzazione Attiva e Gestione dello Scaricamento (RAM)

L'orchestrazione di molteplici LLM su macchine dotate di soli 8 GB di memoria RAM rappresenta la sfida architetturale più complessa del sistema. L'inizializzazione simultanea di due modelli comporterebbe un'immediata saturazione della memoria di sistema (RAM) e il conseguente crash dell'applicativo (Out of Memory).

Per ovviare a questo limite, la versione 7.1.0 mantiene il meccanismo di deallocazione introdotto in V6.8.0:

- **Unload Esplicito a Due Step:** Mentre le versioni precedenti si affidavano al parametro `keep_alive=0` inviato nell'ultima richiesta, delegando il rilascio della memoria a Ollama in modo asincrono, la release corrente introduce il metodo `explicit_unload()` in `BaseAI`.
- **Meccanica del Rilascio:** Al completamento della Fase 1/3, l'orchestratore invia una chiamata API dedicata (`ollama.generate` con `prompt=""` e `keep_alive=0`) che forza il sistema di inferenza a scaricare immediatamente i pesi del modello.

- **Gestione del Kernel e Polling:** Viene introdotta una pausa fissa (`ram_unload_wait`, default 1.5s) per consentire al kernel di reclamare fisicamente le pagine di memoria mmap.

Successivamente, il modulo `main.py` avvia il ciclo di Active Polling tramite `psutil`. È importante notare che `psutil.virtual_memory().available` su sistemi Linux include correttamente cache e buffer recuperabili. Il thread di esecuzione rimane sospeso finché la RAM libera non supera la soglia di sicurezza, entro il limite definito dal parametro `ram_sync_timeout`.

4.3 Passaggio di Consegne (Handoff) e Salvaguardia del Contesto

La convergenza dei dati (Merge) si realizza iniettando la bozza generata dall'Agente A all'interno del contesto operativo dell'Agente B. Il prompt di passaggio di consegne (Handoff) include direttive di inibizione della verbosità: all'Agente B è fatto divieto di reiterare spiegazioni tecniche già fornite dal predecessore, ottimizzando la sintesi finale.

Contestualmente, è attivo un meccanismo di salvaguardia contro la saturazione della finestra di contesto (Context Window). Se l'output dell'Agente A risulta eccessivamente prolisso, la classe `BaseAI` interviene tramite il metodo `truncate_context()`, che recide l'elaborato in eccesso basandosi sul limite dinamico `pipeline_max_context_chars` definito in `config.py`. Questa astrazione garantisce la stabilità attentiva dell'Agente B anche su hardware con risorse limitate.

Capitolo 5

Chat History e Persistenza Conversazionale (V6.6.0)

5.1 Architettura della Memoria di Sessione

A partire dalla versione 6.6.0, CYA N implementa un sistema di gestione della memoria a breve termine per supportare interazioni multi-turno. La chat history è strutturata come una lista globale di sessione gestita dal modulo `main.py`, composta da una sequenza di dizionari nel formato nativo richiesto dai motori di inferenza (`{role: str, content: str}`).

Per scelta architetturale legata alla privacy e alla velocità, la cronologia non viene persistita su disco: risiede esclusivamente nella RAM e viene azzerata alla chiusura del processo o tramite comando esplicito.

5.2 Meccanismo di Sliding Window

Per garantire la stabilità del sistema su hardware con soli 8 GB di RAM e prevenire l'overflow della Context Window (fissata a 4096 token), la cronologia è regolata da una finestra scorrevole (Sliding Window). Il parametro `max_history_turns` in `config.SYSTEM_SETTINGS` (default: 5) definisce la profondità della memoria. Uno "scambio" completo comprende il messaggio dell'utente e la risposta dell'assistente; pertanto, un'impostazione di 5 turni comporta il mantenimento di un massimo di 10 messaggi. Quando la soglia viene superata, il sistema provvede alla rimozione automatica dei messaggi più obsoleti.

5.3 Integrazione e Iniezione del Contesto

La history viene iniettata dinamicamente in ogni chiamata ai modelli. Questa procedura coinvolge i metodi `resolve()`, `resolve_pipeline_a()` e `resolve_pipeline_b()`.

- **Fusione Few-Shot:** Al fine di evitare conflitti di ruolo tra gli esempi predefiniti e i turni reali della conversazione, il metodo `merge_few_shot()` in `BaseAI` fonde gli esempi few-shot direttamente nel System Prompt. Questo garantisce che gli esempi virtuosi fungano da istruzione di base senza alterare la cronologia temporale dei messaggi.
- **Esclusione del Critic Pass:** Per design deliberato, il metodo `execute_critic_pass()` non riceve la cronologia. La revisione critica deve

restare focalizzata esclusivamente sulla coerenza tra la bozza corrente e la richiesta originale dell'utente, evitando che il contesto pregresso possa distorcere o "ammorbidire" la severità della validazione.

5.4 Integrità dello Stato e Controllo Utente

Il sistema protegge la qualità della memoria conversazionale tramite un aggiornamento condizionale: la funzione `_update_history()` viene invocata esclusivamente se la risposta prodotta non è classificata come errore di sistema (`_is_error`). Questo impedisce che messaggi diagnostici (es. "OOM — RAM insufficiente") vengano memorizzati, preservando la coerenza logica dei turni successivi.

In aggiunta, quando viene rilevato un cambio di dominio (`domain_switched = True`), la history viene passata all'agente corrente come lista vuota (`effective_history = []`), isolando il contesto del nuovo dominio da eventuali risposte precedenti provenienti da un dominio diverso. Questo previene la "contaminazione" del contesto nei context switch.

L'utente dispone inoltre del comando speciale `/reset` (o `/clear`) a runtime. L'esecuzione di tale comando svuota istantaneamente la chat history e azzerava lo stato dello Sticky Routing, permettendo di iniziare una nuova sessione tematica senza dover riavviare l'intera infrastruttura.

Capitolo 6

Domain Retention e Sticky Routing (V7.2)

6.1 Motivazione Architettuale

In una conversazione multi-turno fluida, è fisiologico che l'utente formuli query brevi di follow-up (es. "E in Python?", "Dimmi di più", "Fammi un esempio pratico"). Queste interrogazioni, se isolate, non contengono elementi sufficienti per classificare univocamente il dominio.

Senza un meccanismo di ritenzione, tali query verrebbero sistematicamente deflesse verso il dominio general, rompendo irrimediabilmente la continuità semantica della sessione e causando un degrado dell'esperienza utente. Per risolvere questo problema, l'architettura ha introdotto lo Sticky Routing (o Domain Retention), ora nella versione V7.2.

6.2 Stato di Sessione: `last_active_domain`

La continuità è garantita da una variabile di sessione denominata `last_active_domain`, gestita all'interno di `main.py`. Questa variabile traccia l'orizzonte semantico della conversazione e viene aggiornata a ogni risposta conclusa con successo (ossia se `not _is_error(result)`), indipendentemente dal dominio — incluso `general`. Il reset viene eseguito automaticamente nei domain switch tramite l'isolamento della history.

Per le esecuzioni mono-dominio, `last_pipeline_domains` viene azzerato a `('', '')`. Per le pipeline, viene aggiornato con la coppia `(domain_a, domain_b)` per consentire il rilevamento del pipeline follow-up nel turno successivo.

6.3 Logica Decisionale: `_should_sticky_route()`

La determinazione dell'applicazione del Domain Retention è affidata alla funzione `_should_sticky_route()`. Questa routine riceve la query, i domini rilevati, la confidenza, l'ultimo dominio attivo, il flag `is_followup_llm` e la coppia dell'ultima pipeline attiva. Restituisce una tupla `(should_stick, sticky_domain, reason, override_target)`.

La logica si articola in tre step sequenziali (V7.2 — LLM-first):

- **Step 1 — Override (Context Switch):** Se il router rileva un dominio tecnico *diverso* da quello attivo con confidenza superiore alla soglia (`sticky_tech_switch_min` per query lunghe,

`sticky_short_override_min` per query corte), lo sticky viene interrotto e l'override viene segnalato. Questo è il segnale più forte: riconosce che l'utente ha introdotto un netto cambio di argomento.

- **Step 2 — LLM is_followup=True:** Se il router LLM ha classificato la query come follow-up diretto (`is_followup_llm=True`), il routing viene forzato verso l'ultimo dominio attivo. Il LLM è il segnale primario per questo step, eliminando la necessità di euristiche Python basate sulla lunghezza della query.
- **Step 3 — Pipeline Follow-up (Python-side):** Gestisce il caso in cui l'ultima interazione fosse una pipeline e la query attuale si riferisce implicitamente al dominio A della pipeline (non visibile al LLM). Se `last_pipeline_domains` è valorizzato, `top_domain == 'general'` e la query contiene keyword del dominio A, lo sticky viene attivato verso il dominio A.

La funzione non si attiva mai se `last_active_domain` è vuoto o se la sessione si trovava in stato `general` al momento della valutazione degli step 2 e 3.

6.4 Parametri di Configurazione e Fallback

Il comportamento del sistema è finemente tarato tramite i parametri presenti in `config.SYSTEM_SETTINGS`:

- `sticky_short_words` (default 10): soglia di parole per query "corta".
- `sticky_tech_switch_min` (default 0.38): confidenza minima per context switch su query lunghe.
- `sticky_short_override_min` (default 0.65): confidenza minima per context switch su query corte. Più alta di `sticky_tech_switch_min`: su query corte il follow-up è più probabile del context switch, quindi serve un segnale più forte per forzare lo switch.

Interazione con il Fallback a Keyword: La solidità del costrutto è garantita anche in situazioni di emergenza. Qualora il router LLM risulti offline (`class_id == -1`), costringendo il sistema ad affidarsi al dispatcher testuale, la funzione `_should_sticky_route()` viene comunque consultata (con `is_followup_llm=False`). Se il fallback testuale approda a general a causa della brevità della query, il Domain Retention interviene ripristinando il routing corretto, assicurando coerenza tra l'approccio LLM e quello euristico.

Capitolo 7

Critic Pass, Analisi Strutturata e Sanificazione dell'Output

7.1 Revisione Critica: Validazione Antagonistica (Critic Pass)

Nei sistemi ad agenti sequenziali, sussiste il rischio architetturale noto come "Fiducia Cieca" (Blind Trust), fenomeno per cui il nodo finale assimila e riproduce acriticamente i dati forniti dal nodo precedente, propagando eventuali allucinazioni semantiche.

Per interrompere questa catena, CYA N implementa una fase di convalida autonoma denominata Critic Pass. Sfruttando la permanenza in RAM dell'Agente B, l'orchestratore gli impone di rianalizzare il proprio output finale prima della renderizzazione a schermo. Il prompt di validazione (`PIPELINE_PROMPTS['critic']`) inietta dinamicamente la stringa `{original_query}` formulata dall'utente. Questa iniezione funge da ancoraggio semantico: l'LLM è costretto a confrontare la sua sintesi con l'intento originale, verificando la correttezza fattuale dell'integrazione e colmando eventuali deviazioni.

Le direttive impongono inoltre la correzione silente: se non vengono rilevate anomalie, l'agente restituisce il testo invariato astenendosi dal produrre metadati valutativi (es. "Il testo risulta corretto").

7.2 Parsing Strutturato e Gestione dei Tag di Ragionamento

L'adozione di modelli analitici avanzati (come deepseek-r1) comporta l'emissione di token di ragionamento latente racchiusi all'interno di tag strutturati (es. `<think>`). Nella versione attuale, la logica di parsing a livello di streaming in `ai_engine.py` è stata astratta: i tag non sono più codificati rigidamente, ma derivati dalle variabili `think_open_tag` e `think_close_tag` in `config.SYSTEM_SETTINGS`.

Risoluzione Debito Tecnico su helper.py (V6.7.2): Il difetto documentato nelle versioni precedenti — la regex hardcoded `<think>.*?</think>` in `clean_response()` — è stato definitivamente eliminato. La funzione legge ora i tag di apertura e chiusura dinamicamente da `config.SYSTEM_SETTINGS` tramite `re.escape()`. La modifica di `config.py` è ora sufficiente per supportare qualsiasi modello con sintassi di reasoning alternativa. Nessun aggiornamento manuale di `helper.py` è più necessario.

Sanificazione Code-Block Aware (V6.7.2): Oltre alla rimozione dei tag, la routine di sanificazione opera conversioni sui simboli LaTeX e sui logogrammi asiatici (CJK). La funzione `clean_response()` è stata resa pienamente consapevole della struttura

Markdown. Il testo viene diviso tramite una regex sui delimitatori backtick. Le sostituzioni LaTeX → Unicode e il filtro CJK vengono applicati esclusivamente alle sezioni di testo discorsivo, preservando intatti i blocchi di codice. Questo previene la corruzione di snippet tecnici contenenti simboli matematici o caratteri orientali.

Il filtro CJK è controllato dal flag `cjk_filter_enabled` in `SYSTEM_SETTINGS` (default True; impostare False per abilitare caratteri asiatici nei code block).

7.3 Intercettazione delle Eccezioni a Basso Livello

L'architettura rafforza la gestione degli stati di anomalia (Error Handling). Se durante le operazioni di pre-flight la funzione `check_resources()` rileva indisponibilità critica di RAM non risolvibile, il motore `ai_engine.py` solleva l'eccezione tipizzata `ResourceExhaustedError`.

L'intercettazione di questo errore nel blocco try-except di `main.py` blocca istantaneamente il task corrente, impedendo all'orchestrazione di passare un messaggio di errore generico all'Agente B come se fosse una bozza valida da elaborare, garantendo così una chiusura controllata ed elegante (Graceful Degradation).

Capitolo 8

Topologia della Flotta LLM e Parametri di Inferenza

8.1 Provisioning e Gerarchia dei Modelli

L'infrastruttura di CYA N non converge su un modello generalista monolitico, bensì su una topologia a sciame (Swarm Topology) composta da modelli locali quantizzati, definiti dinamicamente nel modulo `config.py`. La distribuzione del carico cognitivo è organizzata secondo una gerarchia di competenza e requisiti hardware. La flotta include anche un router dedicato (`qwen2.5:1.5b`) per la classificazione semantica, scaricato prima dell'attivazione degli agenti.

- **Dominio Informatica (Coding):** Il nodo esecutivo primario è `qwen2.5:9b`. Per garantire la resilienza su macchine a basse prestazioni, il sistema prevede un fallback hardware: se la RAM disponibile rilevata scende sotto la soglia medium (5.5 GB), l'istanziamento viene deviata sul modello `qwen2.5-coder:1.5b`, ottimizzato per spazi di memoria estremamente ristretti.
- **Dominio Logico-Matematico (Math):** La risoluzione di problematiche inerenti l'analisi complessa e la statistica è affidata in via esclusiva a `deepseek-r1:7b`. Questo dominio non ammette archi di fallback: l'uso di modelli con un computo di parametri inferiore provocherebbe un degrado inaccettabile e allucinazioni nel ragionamento logico.
- **Domini Normativo e Generalista (Rights e General):** Per coprire la vasta mole di giurisprudenza e cultura generale, l'architettura fa affidamento su `gpt-oss:20b`. Considerando l'imponente richiesta di memoria, sussiste un controllo di sbarramento a 12 GB di RAM (large); al di sotto di tale limite, il traffico viene reindirizzato in sicurezza verso il più snello `llama3.2:3b`.

***Nota sulla Configurazione di Sviluppo:** La versione di sviluppo del progetto opera su hardware con 8 GB di RAM. Per consentire la verifica funzionale della pipeline senza dipendere dai tempi di caricamento dei modelli da 7B o superiori, il file `config.py` nella build di sviluppo utilizza modelli a bassa impronta (`qwen2.5-coder:1.5b`) per tutti i domini. La topologia a sciame completa è quella di riferimento per il deployment su workstation con almeno 16 GB di RAM dedicata.*

8.2 Calibrazione delle Temperature (Determinismo vs Entropia)

Il parametro di Temperatura (T) modula la distribuzione di probabilità dei token in fase di inferenza, determinando il bilanciamento tra rigore analitico e creatività discorsiva. L'architettura assegna indici termici differenziati per ciascun dominio operativo:

- **$T = 0.2$ (Math):** Impostazione prossimo-deterministica. Sfruttata per inibire l'entropia semantica, risulta indispensabile per l'enunciazione di formule, dimostrazioni e teoremi, ambiti in cui l'esattezza strutturale non ammette variazioni stocastiche.
- **$T = 0.4$ (Rights):** Entropia vincolata. Fornisce all'agente sufficiente elasticità lessicale per articolare concetti giuridici in prosa, mantenendo un vincolo probabilistico stringente che impedisce la sintesi allucinatoria di testi di legge inesistenti.
- **$T = 0.5$ (Coding):** Modulazione ibrida. Consente l'esplorazione di design pattern e soluzioni algoritmiche laterali, pur vincolando la generazione sintattica per garantire la validità del codice emesso.
- **$T = 0.7$ (General):** Massima varianza autorizzata. Applicato al dominio generalista per sbloccare la naturalezza colloquiale e l'eterogeneità del vocabolario.
- **$T = 0.0$ (Router):** Il micro-modello di classificazione opera a temperatura zero, garantendo classificazioni deterministiche e riproducibili indipendentemente dalla sessione.

Capitolo 9

Istruzioni Operative: Messa in Produzione

9.1 Inizializzazione del Motore di Inferenza (Ollama)

Il prerequisito fondamentale per il dispiegamento dell'infrastruttura di CYA N è l'installazione e la configurazione del motore di inferenza locale. Nel rispetto del vincolo di totale indipendenza dalle reti esterne, l'architettura adotta Ollama come demone locale per l'esecuzione dei modelli. Ollama svolge un duplice ruolo: funge da nodo centrale per l'elaborazione dei tensori dei modelli generativi e fornisce l'API ottimizzata per l'invocazione del router semantico (`qwen2.5:1.5b`).

9.2 Provisioning della Flotta: Scaricamento dei Modelli

Una volta instaurata la comunicazione con il demone locale, è necessario allocare sull'hardware i pesi neurali dei modelli previsti dall'architettura V7.1.0. L'allocazione avviene da terminale tramite i seguenti comandi, che includono sia i nodi primari che quelli di fallback:

- **Router Semantico (dipendenza critica):** `ollama pull qwen2.5:1.5b`
- **Dominio Informatica (Coding):** `ollama pull qwen2.5:9b` e `ollama pull qwen2.5-coder:1.5b`
- **Dominio Logico-Matematico (Math):** `ollama pull deepseek-r1:7b`
- **Domini Rights e General:** `ollama pull gpt-oss:20b` e `ollama pull llama3.2:3b`

9.3 Configurazione dell'Ambiente e Avvio

Il software richiede un ambiente Python 3.8 o superiore. Le dipendenze di sistema includono:

```
pip install psutil ollama sentence-transformers
```

È fondamentale evidenziare la criticità della libreria `psutil`: essa costituisce il modulo di telemetria hardware del sistema. In sua assenza, CYA N perderebbe la capacità di interrogare la memoria RAM a runtime, inibendo i declassamenti preventivi, le procedure di Active Polling e lo scaricamento esplicito (Explicit Unload), esponendo la macchina a un inevitabile blocco per saturazione della memoria.

La libreria `sentence-transformers` è richiesta per il futuro classificatore neurale (`neural_classifier.py`, attualmente in fase di sviluppo nel branch `set_neural_network_routing`).

Completata l'installazione, l'infrastruttura viene avviata dal nodo principale:

```
python code/main.py
```

Capitolo 10

Glossario Architetture

Arco di Fallback Preventivo

Il meccanismo di mitigazione dei guasti (Failover) che intercetta a runtime l'indisponibilità del router LLM o una riduzione critica della RAM e reindirizza le operazioni verso il dispatcher testuale o un modello generativo di taglia inferiore.

Chat History (Sliding Window)

Il buffer di sessione conversazionale gestito da `main.py`. Mantiene gli ultimi `max_history_turns` scambi user/assistant per supportare il dialogo multi-turno. Non è persistita su disco. Viene isolata (svuotata per l'agente corrente) in caso di domain switch.

Critic Pass

Modulo di autovalutazione antagonistica che forza l'agente a validare il proprio output confrontandolo con la richiesta originale per mitigare le allucinazioni.

Domain Retention (Sticky Routing)

Meccanismo di continuità semantica che ancora le query brevi all'ultimo dominio attivo, preservando la coerenza della sessione. Versione V7.2: priorità al flag `is_followup` del router LLM; il Python gestisce solo override e pipeline follow-up.

Explicit Unload

Chiamata API che forza il rilascio immediato del memory mapping dei tensori dopo il completamento dell'Agente A (e del router), riducendo la latenza del polling RAM.

Filtro di Complessità

Validazione booleana che controlla se la query supera la soglia `min_words_for_pipeline` (default 12) prima di attivare la Pipeline. Supportato da promozione score-based.

LLM Router

Modulo di classificazione semantica (`llm_router.py`) basato su micro-LLM (`qwen2.5:1.5b`). Sostituisce il precedente motore k-NN/LanceDB. Espone un'interfaccia compatibile con il futuro `neural_classifier.py`.

Matrice Autoritativa (`pipeline_order_matrix`)

Struttura gerarchica che impone la sequenza esecutiva corretta tra i domini indipendentemente dai punteggi del router.

P0 GENERAL Guard

Routine di isolamento che neutralizza i falsi positivi generati dal dominio generalista nelle query ibride, degradando la pipeline a mono-dominio tecnico.

Pipeline Follow-up

Meccanismo Python-side dello Sticky Routing (Step 3) che rileva query di follow-up relative al dominio A dell'ultima pipeline eseguita, non visibile al router LLM.

Stateful (Con stato)

Paradigma in cui le istanze dei modelli vengono preservate "calde" in RAM per ottimizzare la latenza delle interazioni successive.